

TASK 03 — Idempotent APIs & Duplicate Request Handling

Step 1 — Understand idempotency

What is idempotency

An operation is **idempotent** when performing it multiple times has the same effect as performing it once.

For example:

Request 1: Create Order → Order #1001 created

Request 2: Same request → Don't create another order

Return Order #1001

Request 3: Same request → Don't create another order

Return Order #1001

Without idempotency

Suppose the customer clicks:

"Place Order"

The request reaches the server:

Client

| POST /api/v1/orders

Order Service

|

Create Order #1001

But the network is slow.

The client doesn't receive the response and retries:

Client

| POST /api/v1/orders

Order Service

|

Create Order #1002

Now the customer has **two orders**.

Why do retries create duplicate operations

Retries can happen because of:

1. Network timeout
2. Slow server response
3. Connection failure
4. Client retry logic
5. Load balancer timeout
6. Mobile network problems
7. User clicking the button multiple times

For example:

POST /api/v1/orders

The client sends:

```
{  
  "productId": 101,  
  "quantity": 2  
}
```

The server successfully creates the order.

But the response doesn't reach the client.

The client retries.

Now the server receives the same request again.

Without protection:

Order #1 → Created

Order #2 → Created

This is the problem that **Idempotency-Key** solves.

Real-world payment/order examples

Consider a payment.

The customer wants to pay:

₹1,000

The request is:

POST /api/v1/payments

The payment succeeds, but the response is lost.

The application retries.

Without idempotency:

Payment 1 → ₹1,000

Payment 2 → ₹1,000

Customer is charged:

₹2,000 ❌

With idempotency:

Payment 1 → ₹1,000

Payment 2 → Same payment result

Customer is charged:

₹1,000 ✅

This is why idempotency is particularly important for:

- Payments
- Orders
- Money transfers
- Ticket booking
- Account creation
- Subscription creation
- Resource creation

Step 2 — Select an API

POST /api/v1/orders

We will create this API.

Example request:

POST /api/v1/orders

Content-Type: application/json

Idempotency-Key: ORD-123456

Request body:

```
{  
  "productId": 101,  
  "quantity": 2  
}
```

Step 3 — Introduce Idempotency-Key

Idempotency-Key: ORD-123456

Complete request:

POST /api/v1/orders

Idempotency-Key: ORD-123456

Content-Type: application/json

```
{  
  "userId": 101,  
  "productId": 5001,  
  "quantity": 2  
}
```

What is the Idempotency-Key

It is a **unique identifier for one logical operation**.

Think of it like a **receipt number**.

For example: ORD-123456

means:

"This particular order creation operation."

If the client sends the same request again with the same key:

Idempotency-Key: ORD-123456

the server knows:

"I have already processed this operation."

So it should **not create another order**.

Step 4 — Store the Key

You can store this information in **Redis or PostgreSQL**.

For example:

Idempotency Key	User ID	Status	Response
ORD-123456	101	COMPLETED	Order 1001 created

Why store the response

Suppose the first request creates:

Order ID: 1001

and returns:

```
{  
  "orderId": 1001,  
  "status": "CREATED"  
}
```

The client doesn't receive the response because of a network problem.

The client retries:

Idempotency-Key: ORD-123456

The server checks the database/Redis:

ORD-123456 → Already processed

Instead of creating another order, it returns the **previous response**:

```
{  
  "orderId": 1001,  
  "status": "CREATED"  
}
```

So the result remains the same.

Complete Flow

The complete process looks like this:

Client

| POST /api/v1/orders

| Idempotency-Key: ORD-123456

API Gateway / Order Service

| Check Idempotency-Key

Redis / PostgreSQL

| Is ORD-123456 already present?

+---- NO ----> Process Order

| |

| v

| Create Order

| |

| v

| Save key + response

| |

| v

| Return Order 1001

|

+---- YES ---> Return saved response

What happens during a retry

First request

POST /api/v1/orders

Idempotency-Key: ORD-123456

Server checks:

ORD-123456 → Not found

So it creates the order:

Order ID = 1001

Then stores:

Key: ORD-123456

Status: COMPLETED

Response: Order 1001

Second request

The client retries:

POST /api/v1/orders

Idempotency-Key: ORD-123456

Server checks Redis/database:

ORD-123456 → Found

Therefore:

DO NOT CREATE ANOTHER ORDER

Return the original result:

```
{  
  "orderId": 1001,  
  "status": "CREATED"  
}
```

Real-world example — Payment

Imagine you are paying ₹1,000.

You click:

Pay Now

The payment request reaches the payment server.

The server successfully charges:

₹1,000

But the response doesn't reach your phone because of a network problem.

Your application thinks:

"Payment failed."

You click **Pay Now** again.

Without idempotency:

Payment 1 → ₹1,000

Payment 2 → ₹1,000

Total = ₹2,000 ❌

With idempotency:

Payment 1 → ₹1,000

Payment 2 → Same payment result

Total = ₹1,000 ✅

Real-world example — Order

Suppose you purchase:

iPhone

Quantity: 1

Request:

POST /api/v1/orders

Idempotency-Key: ORDER-789

First request:

Order created → 5001

Retry:

POST /api/v1/orders

Idempotency-Key: ORDER-789

Server finds:

ORDER-789 → Order 5001

It returns:

```
{  
  "orderId": 5001,  
  "status": "CREATED"  
}
```

It **doesn't** create Order 5002.

Redis Example

A Redis record could look conceptually like:

idempotency: ORD-123456

with:

```
{  
  "userId": 101,  
  "status": "COMPLETED",  
  "response": {  
    "orderId": 1001,  
    "status": "CREATED"  
  },  
  "createdAt": "2026-10-07T09:30:00"  
}
```

Redis is useful because checking a key is very fast.

You can also use PostgreSQL when you need durable database storage.

Important Status Values

A practical implementation can use:

PROCESSING

COMPLETED

FAILED

PROCESSING

The request is currently being handled.

ORD-123456 → PROCESSING

If another identical request arrives, you can prevent two workers from processing it simultaneously.

COMPLETED

The operation finished successfully.

ORD-123456 → COMPLETED

Return the stored response.

FAILED

The operation failed.

Depending on your design, you may allow the same key to be retried or keep the failure result. The important point is to define this behavior explicitly.

Step 5 — Process first request

Request

The client sends a request to create an order.

Example:

POST /api/orders

Idempotency-Key: ABC123

Request body:

```
{  
  "productId": 101,  
  "quantity": 2  
}
```

The important part is:

Idempotency-Key: ABC123

This key uniquely identifies this particular operation.

2. Check the key

The server checks its database or Redis:

Does ABC123 already exist?

At this point:

ABC123 → Not Found

So this is a **new request**.

3. Create the order

Because the key doesn't exist, the server creates the order.

For example:

Order ID = 5001

Product = 101

Quantity = 2

4. Store the result

After creating the order, the server stores the idempotency key and the response.

For example:

Key: ABC123

Result:

```
{  
  "orderId": 5001,  
  "status": "CREATED"  
}
```

This is important because the server can use this stored result if the same request arrives again.

Step 6 — Process duplicate request

The client sends: POST /api/orders

Idempotency-Key: ABC123

again.

The server checks: ABC123 → EXISTS

The server understands:

"This request has already been processed."

Therefore, it **does not create another order**.

Instead, it returns the previously stored result:

```
{  
  "orderId": 5001,  
  "status": "CREATED"  
}
```

So the result is:

First request → Order 5001 created

Second request → Order 5001 result returned

Third request → Order 5001 result returned

Only **one order** is created.

Imagine a customer clicks the **Pay** button twice because the internet is slow.

Without idempotency:

Click 1 → Create Order 5001

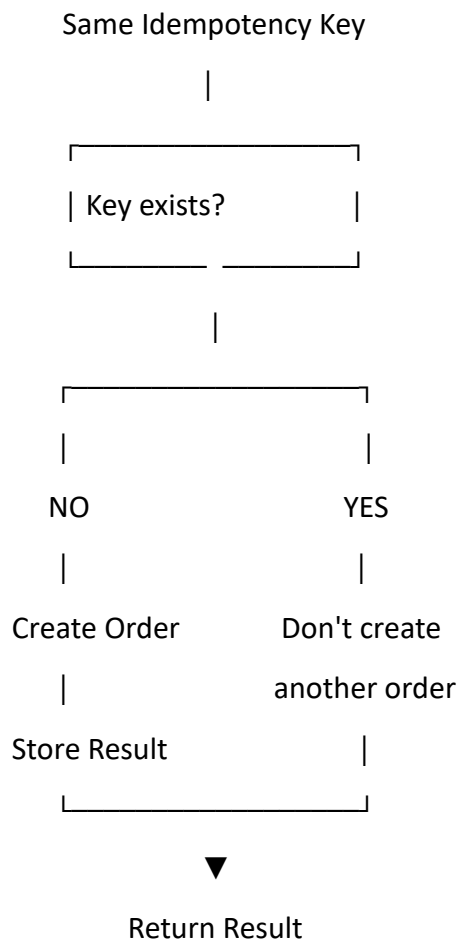
Click 2 → Create Order 5002

The customer may accidentally get **two orders / two payments**.

With idempotency:

Click 1 → Create Order 5001

Click 2 → Detect duplicate → Return Order 5001



Step 7 — Test duplicate requests

send this request 3 times:

POST /api/orders

Idempotency-Key: ABC123

Expected behavior

Request 1:

ABC123 not found



Create Order



Order ID: 5001



Store result

Request 2:

ABC123 found



Don't create order



Return Order 5001

Request 3:

ABC123 found



Don't create order



Return Order 5001

Final database result

Only one order should exist:

Orders

ID	Product	Quantity
----	---------	----------

5001	101	2
------	-----	---

Not:

5001

5002

5003

Simple real-world example

Think of an **ATM withdrawal**.

You request:

Withdraw ₹1,000

Transaction Key = TXN123

The bank processes it:

TXN123 → ₹1,000 withdrawn

If the network fails and your application sends the same request again:

TXN123

The bank checks the transaction key and says:

TXN123 already processed

It **doesn't withdraw another ₹1,000**.

That's the main purpose of **idempotency**.

Step 8 — Verify only one order is created

After sending the same request multiple times, check the database.

Example:

Request 1

↓

Idempotency-Key: ABC123

↓

Order 5001 created

Then the same request is sent again:

Request 2

↓

Idempotency-Key: ABC123



Existing key found



Do NOT create another order

Expected database result

Orders

Order ID

5001

There should **not** be:

5001

5002

5003

One idempotency key = one order creation.

Step 9 — Test retry after timeout

Client sends request

The client sends:

POST /api/orders

Idempotency-Key: ABC123

For example:

```
{  
  "productId": 101,  
  "quantity": 2  
}
```

Order is created

The server successfully processes the request:

ABC123 → Create Order 5001

So the order is actually created.

Response is lost

Suppose there is a network problem.

The server created the order, but the response never reaches the client.

For example:

Server → Order 5001 created

Server → Response

X

Network failure

X

Client → Does not receive response

The client may think:

"Maybe the order was not created."

Client retries

The client sends the request again:

POST /api/orders

Idempotency-Key: ABC123

The **same idempotency key** is very important.

The server checks: Does ABC123 exist?

It finds:

ABC123 → EXISTS

Order → 5001

Therefore, it does **not** create Order 5002.

Instead, it returns the original result:

```
{  
  "orderId": 5001,  
  "status": "CREATED"  
}
```


CLIENT

| Request

| Key = ABC123

API SERVER

|

Create Order

|

Order 5001

| Response

X

Response Lost

|

CLIENT

| Retry

| Key = ABC123

API SERVER

|

Check Idempotency Key

|

Key EXISTS

|

Do NOT create order

|

Return Order 5001

Without idempotency

First request → Order 5001

Retry request → Order 5002

With idempotency

First request → Order 5001

Retry request → Return Order 5001

Step 10 — Document the idempotency design

After implementing and testing the feature, document **how the idempotency system works**.

1. Idempotency key

Explain where the client sends the key.

Example:

Idempotency-Key: ABC123

Explain that the key identifies a particular operation.

2. Storage

Document where the key and result are stored.

For example:

Redis

or

Database

Example Redis data:

Key:

idempotency:ABC123

Value:

```
{  
  "orderId": 5001,  
  "status": "CREATED"  
}
```

3. First request behavior

Document:

Key doesn't exist

↓

Process request



Create order



Store result



Return response

4. Duplicate request behavior

Document:

Key exists



Don't process again



Return stored result

5. Retry behavior

Explain what happens if the client doesn't receive the response.

Request



Order created



Response lost



Client retries



Same idempotency key



Existing result found



Return original result